Summarize

Chapter 3. Architectural Modularity

Tuesday, September 21 09:33



It was the same conference room they had been in a hundred times before, but today the atmosphere was different. Very different. As people gathered, no small talk was exchanged. Only silence. The sort of dead silence that you could cut with a knife. Yes, that was indeed an appropriate cliche given the topic of the meeting.

The business leaders and sponsors of the failing Sysops Squad ticketing application met with the application architects, Addison and Austen, with the purpose of voicing their concern and frustration about the inability of the IT department to fix the never-ending issues associated with the trouble ticket application. "Without a working application," they had said, "we cannot possibly continue to support this business line."

As the tense meeting ended, the business sponsors quietly filed out one by one, leaving Addison and Austen alone in the conference room.

"That was a bad meeting," said Addison. "I can't believe they're actually blaming *us* for all the issues we're currently facing with the trouble ticket application. This is a really bad situation."

"Yeah, I know," said Austen. "Especially the part about possibly closing down the product support business line. We'll be assigned to other projects, or worse, maybe even let go. Although I'd rather be spending all of my time on the soccer field or on the slopes skiing in the winter, I really can't afford to lose this job."

"Neither can I," said Addison. "Besides, I really like the development team we have in place, and I'd hate to see it broken up."

"Me too," said Austen. "I still think breaking apart the application would solve most of these issues."

“I agree with you,” said Addison, “but how do we convince the business to spend more money and time to refactor the architecture? You saw how they complained in the meeting about the amount of money we’ve already spent applying patches here and there, only to create additional issues in the process.”

“You’re right,” Austen said. “They would never agree to an expensive and time-consuming architecture migration effort at this point.”

“But if we both agree that we need to break apart the application to keep it alive, how in the world are we going to convince the business and get the funding and time we need to completely restructure the Sysops Squad application?” asked Addison.

“Beats me,” said Austen. “Let’s see if Logan is available to discuss this problem with us.”

Addison looked online and saw that Logan, the lead architect for Penultimate Electronics, was available. Addison sent a message explaining that they wanted to break apart the existing monolithic application, but weren’t sure how to convince the business that this approach would work. Addison explained in the message that they were in a real bind and could use some advice. Logan agreed to meet with them and joined them in the conference room.

“What makes you so sure that breaking apart the Sysops Squad application will solve all of the issues?” asked Logan.

“Because,” said Austen, “we’ve tried patching the code over and over, and it doesn’t seem to be working. We still have way too many issues.”

“You’re completely missing my point,” said Logan. “Let me ask you the question a different way. What assurances do you have that breaking apart the system will accomplish anything more than just spending more money and wasting more valuable time?”

“Well,” said Austen, “actually, we don’t.”

“Then how do you know breaking apart the application is the right approach?” asked Logan.

“We already told you,” said Austen, “because nothing else we try seems to work!”

“Sorry,” said Logan, “but you know as well as I do that’s not a reasonable justification for the business. You’ll never get the funding you need with that kind of reason.”

“So, what would be a good business justification?” asked Addison. “How do we sell this approach to the business and get the additional funding approved?”

"Well," said Logan, "to build a good business case for something of this magnitude, you first need to understand the benefits of architectural modularity, match those benefits to the issues you are facing with the current system, and finally analyze and document the trade-offs involved with breaking apart the application."

Businesses today face a torrent of change; market evolution seems to keep accelerating at a blistering pace. Business drivers (such as mergers and acquisitions), increased competition in the marketplace, increased consumer demand, and increased innovation (such as automation through machine learning and artificial intelligence) necessarily require changes to underlying computer systems. In many cases, those changes in computer systems consequently necessitate changes to the underlying architectures supporting them.

However, it's not only business that's undergoing constant and rapid change—it's also the technical environment in which those computer systems reside. Containerization, the move to cloud-based infrastructure, the adoption of DevOps, and even new advancements in continuous delivery pipelines all impact the underlying architecture of those computer systems.

It's difficult in today's world to manage all of this constant and rapid change with respect to software architecture. Software architecture is the foundational structure of a system, and is therefore generally thought of as something that should remain stable and not undergo frequent change, similar to the underlying structural aspects of a large building or skyscraper. However, unlike the structural architecture of a building, software architecture must constantly change and adapt to meet the new demands of today's business and technology environment.

Consider the increased number of mergers and acquisitions happening in today's marketplace. When one company acquires another, not only does it acquire the physical aspects of a company (such as people, buildings, inventory, and so on) but also more customers. Can the existing systems in either company scale to meet the increase in user volume as a result of the merger or acquisition? Scalability is a big part of mergers and acquisitions, as is agility and extensibility, all of which are *architectural concerns*.

Large monolithic (single deployment) systems generally do not provide the level of scalability, agility, and extensibility required to support most mergers and acquisitions. The capacity for additional machine resources (threads, memory, and CPU) fills up very quickly. To illustrate this point, consider the water glass shown in Figure 3-1. The glass represents the server (or virtual machine), and the water represents the application. As monolithic applications grow to handle increased consumer demand and user load (whether from mergers, acquisitions, or company growth), they begin to consume more and more resources. As more water is added to the glass (representing the growing monolithic application), the glass be-

gins to fill up. Adding another glass (represented as another server or virtual machine) does nothing, because the new glass would contain the same amount of water as the first one.



Figure 3-1. A full glass representing a large monolithic application close to capacity

One aspect of architectural modularity is breaking large monolithic applications into separate and smaller parts to provide more capacity for further scalability and growth, while at the same time facilitating constant and rapid change. In turn, these capabilities can help achieve a company's strategic goals.

By adding another empty glass to our water glass example and breaking the water (application) into two separate parts, half the water can now be poured into the new empty glass, providing 50% more capacity, as shown in Figure 3-2. The water glass analogy is a great way of explaining architectural modularity (the breaking up of monolithic applications) to business stakeholders and C-level executives, who will inevitably be paying for the architecture-refactoring effort.

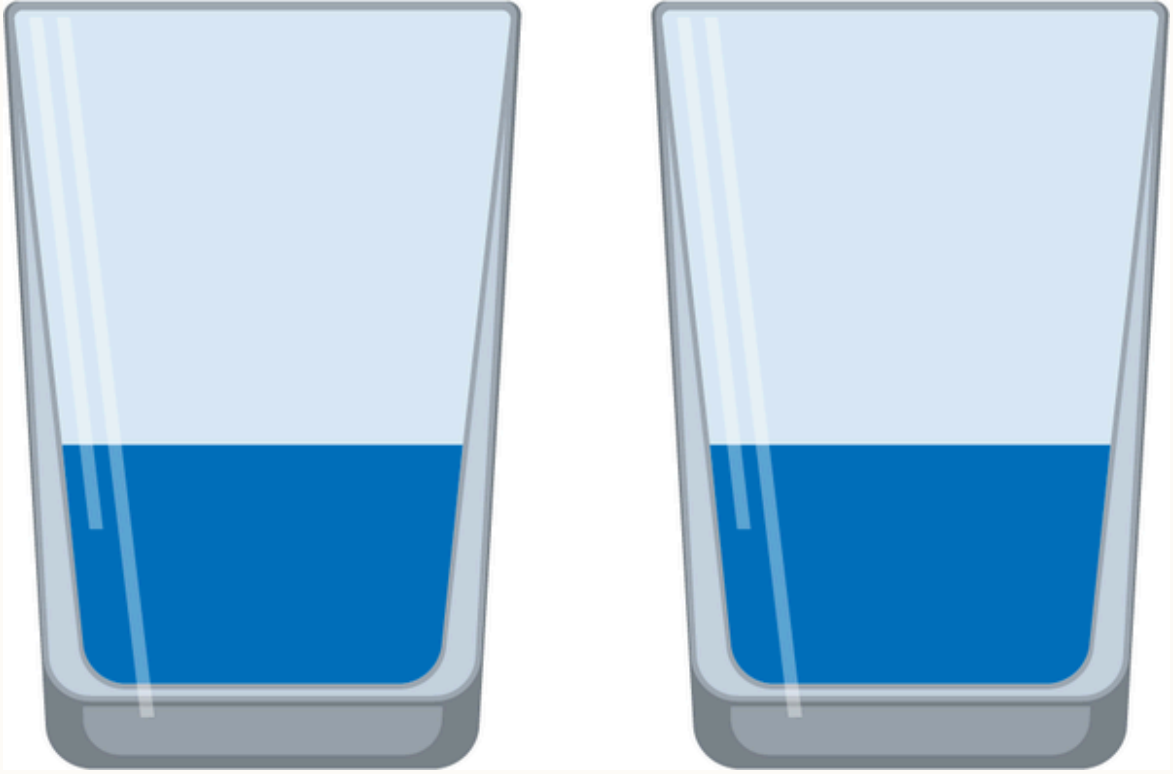


Figure 3-2. Two half-full glasses representing an application broken apart with plenty of capacity for growth

Increased scalability is only one benefit of architectural modularity. Another important benefit is *agility*, the ability to respond quickly to change. An article from *Forbes* in January 2020 by David Benjamin and David Komlos stated the following:

> *There is one thing that will separate the pack into winners and losers: the on-demand capability to make bold and decisive course-corrections that are executed effectively and with urgency.*

Businesses must be agile in order to survive in today's world. However, while business stakeholders may be able to make quick decisions and change direction quickly, the company's technology staff may not be able to implement those new directives fast enough to make a difference. Enabling technology to move as fast as the business (or, conversely, preventing technology from slowing the business) requires a certain level of architectural agility.

Modularity Drivers

Architects shouldn't break a system into smaller parts unless clear business drivers exist. The primary business drivers for breaking applications into smaller parts include *speed-to-market* (sometimes called time-to-market) and achieving a level of *competitive advantage* in the marketplace.

Speed-to-market is achieved through architectural agility—the ability to respond quickly to change. Agility is a compound architectural characteristic made up of many other architec-

ture characteristics, including maintainability, testability, and deployability.

Competitive advantage is achieved through speed-to-market combined with scalability and overall application availability and fault tolerance. The better a company does, the more it grows, hence the need for more scalability to support increased user activity. *Fault tolerance*, the ability of an application to fail and continue to operate, is necessary to ensure that as parts of the application fail, other parts are still able to function as normal, minimizing the overall impact to the end user. Figure 3-3 illustrates the relationship between the technical drivers and the resulting business drivers for modularity (enclosed within boxes).

Businesses must be agile to survive in today's fast-paced and ever-changing volatile market, meaning the underlying architectures must be agile as well. As illustrated in Figure 3-3, the five key architectural characteristics to support agility, speed-to-market, and, ultimately, competitive advantage in today's marketplace are availability (fault tolerance), scalability, deployability, testability, maintainability.

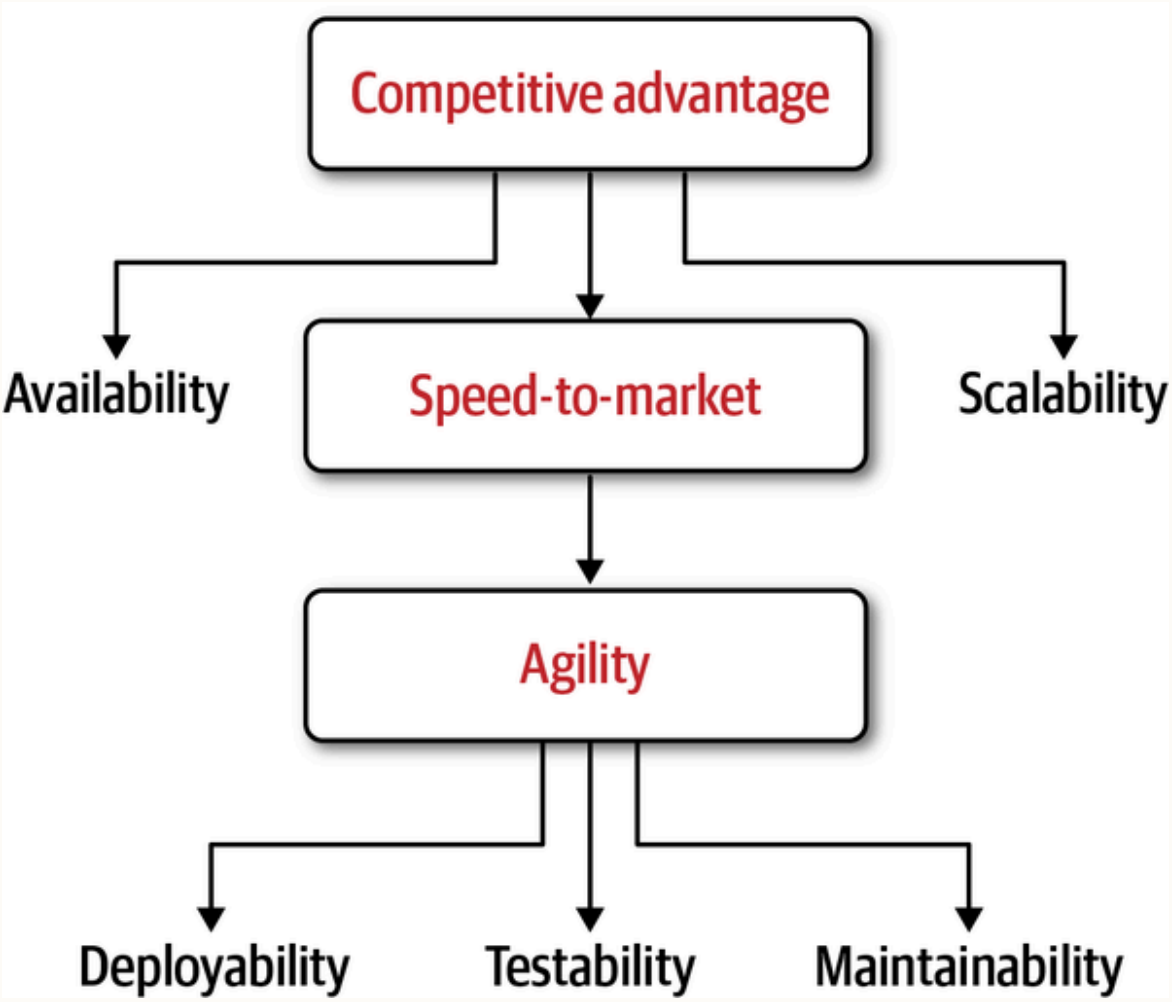


Figure 3-3. The drivers for modularity and the relationships among them

Note that *architectural modularity* does not always have to translate to a distributed architecture. Maintainability, testability, and deployability (defined in the following sections) can also be achieved through monolithic architectures such as a modular monolith or even a microkernel architecture (see Appendix B for a list of references providing more informa-

tion about these architecture styles). Both of these architecture styles offer a level of architectural modularity based on the way the components are *structured*. For example, with a modular monolith, components are grouped into well-formed domains, providing for what is known as a *domain partitioned architecture* (see *Fundamentals of Software Architecture*, Chapter 8, page 103). With the microkernel architecture, functionality is partitioned into separate plug-in components, allowing for a much smaller testing and deployment scope.

Maintainability

Maintainability is about the ease of adding, changing, or removing features, as well as applying internal changes such as maintenance patches, framework upgrades, third-party upgrades, and so on. As with most composite architecture characteristics, maintainability is hard to define objectively. Alexander von Zitzewitz, software architect and founder of hello2morrow, wrote an article about a new metric for objectively defining the maintainability level of an application. While von Zitzewitz's maintainability metric is fairly complicated and involves lots of factors, its initial form is as follows:

$$ML = 100 * \sum_{i=1}^{k} c_i$$

where *ML* is the maintainability level of the overall system (percentage from 0% to 100%), *k* is the total number of logical components in the system, and c_i is the coupling level for any given component, with a special focus on incoming coupling levels. This equation basically demonstrates that the higher the incoming coupling level between components, the lower the overall maintainability level of the codebase.

Putting aside complicated mathematics, some of the typical metrics used for determining the relative maintainability of an application based on components (the architectural building blocks of an application) include the following:

Component coupling

: The degree and manner to which components know about one another

Component cohesion

: The degree and manner to which the operations of a component interrelate

Cyclomatic complexity

: The overall level of indirection and nesting within a component

Component size

The number of aggregated statements of code within a component

Technical versus domain partitioning

Components aligned by technical usage or by domain purpose—see Appendix A

Within the context of architecture, we are defining a *component* as an architectural building block of the application that does some sort of business or infrastructure function, usually manifested through a package structure (Java), namespace (C#), or physical grouping of files (classes) within some sort of directory structure. For example, the component Order History might be implemented through a set of class files located in the namespace `app.business.order.history`.

Large monolithic architectures generally have low levels of maintainability due to the technical partitioning of functionality into layers, the tight coupling between components, and weak component cohesion from a domain perspective. For example, consider a new requirement within a traditional monolithic layered architecture to add an expiration date to items contained in a customer's wish list (items in a list to maybe purchase at a later time). Notice in Figure 3-4 that the change scope of the new requirement is at an *application level* since the change is propagated to all of the layers within the application.

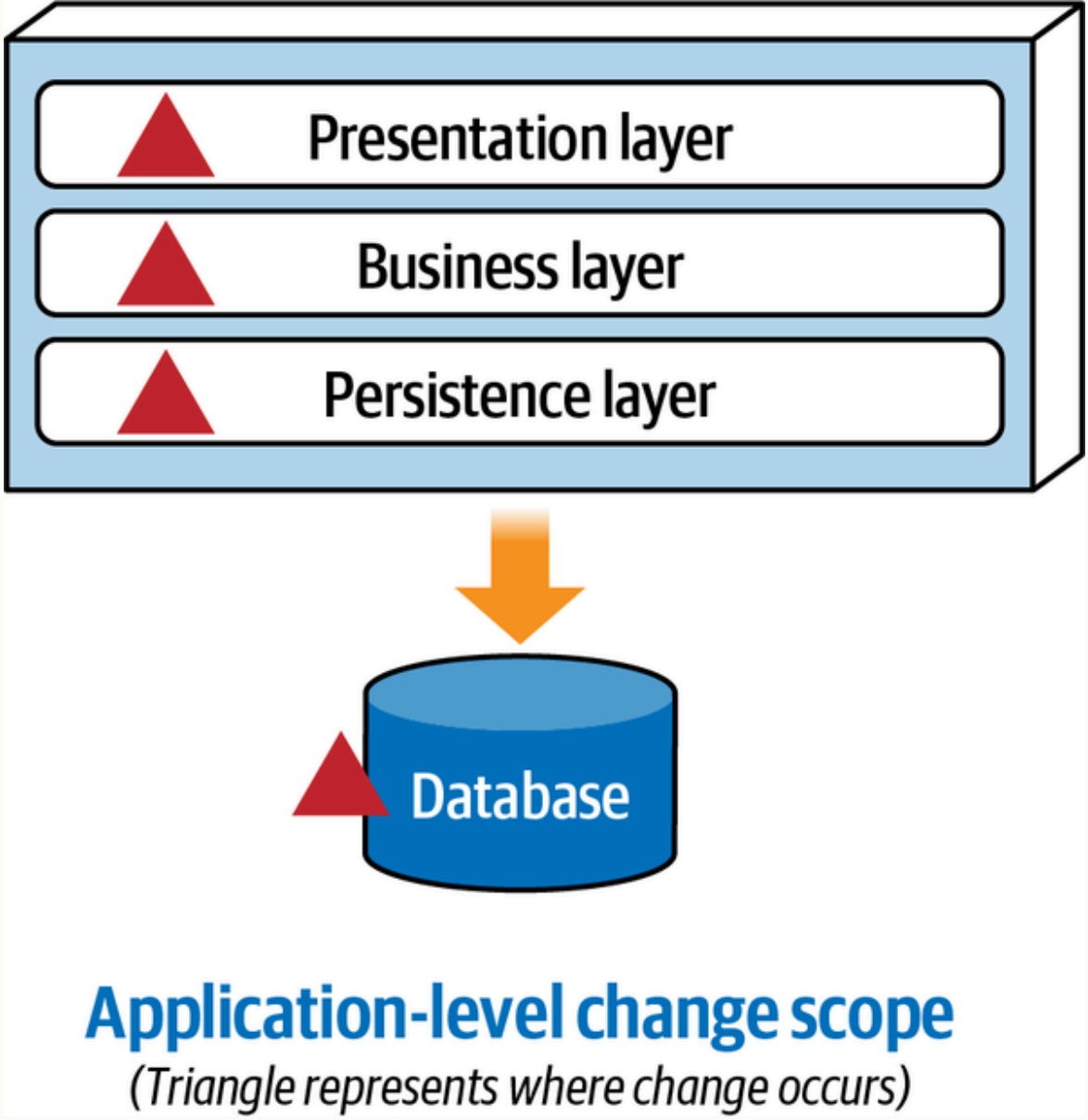


Figure 3-4. With monolithic layered architectures, change is at an application level

Depending on the team structure, implementing this simple change to add an expiration date to wish list items in a monolithic layered architecture could possibly require the coordination of at least three teams:

- A member from the user interface team would be needed to add the new expiry field to the screen.
- A member from the backend team would be needed to add business rules associated with the expiry date and change contracts to add the new expiry field.
- A member from the database team would be needed to change the table schema to add the new expiry column in the Wishlist table.

Since the Wishlist domain is spread throughout the entire architecture, it becomes harder to maintain a particular domain or subdomain (such as Wishlist). Modular architectures, on the other hand, partition domains and subdomains into smaller, separately deployed units of software, thereby making it easier to modify a domain or subdomain. Notice that with a distributed service-based architecture, as shown in Figure 3-5, the change scope of the new

requirement is at a *domain level* within a particular domain service, making it easier to isolate the specific deployment unit requiring the change.

Moving to even more architectural modularity such as a microservices architecture, as illustrated in Figure 3-6, places the new requirement at a *function-level* change scope, isolating the change to a specific service responsible for the wish list functionality.

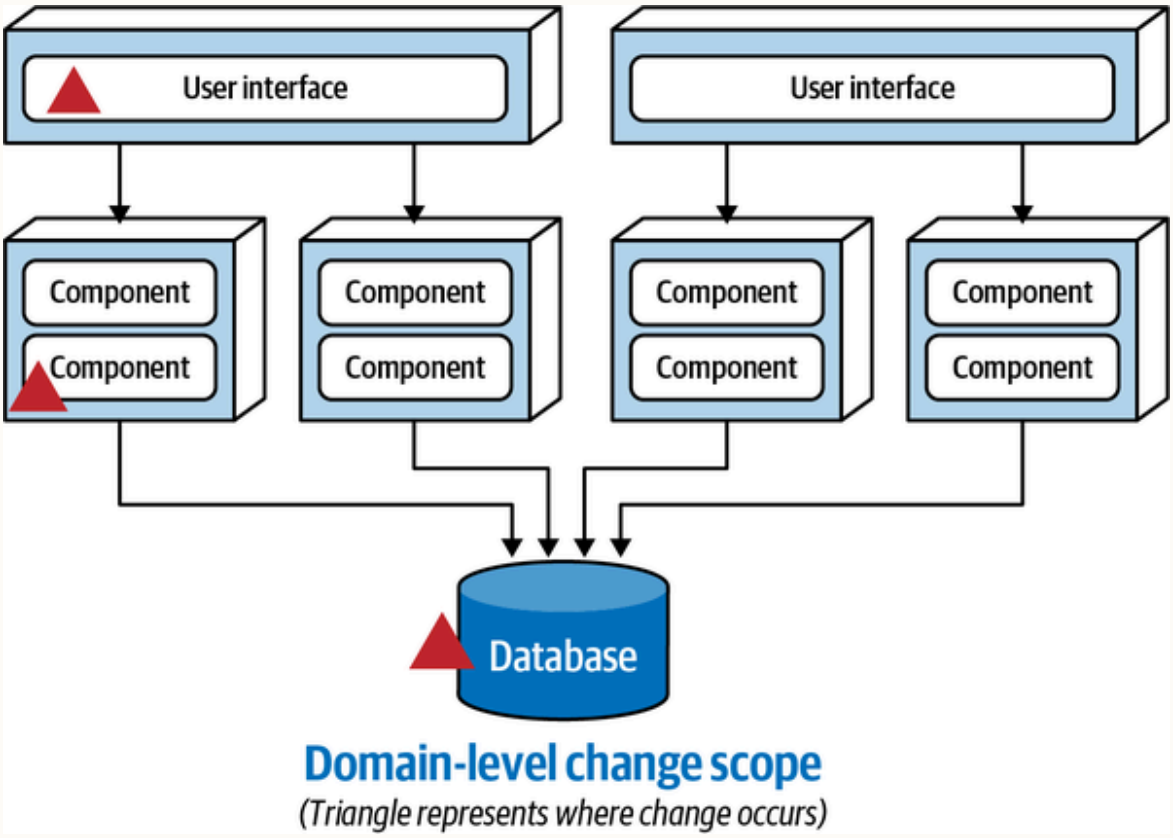


Figure 3-5. With service-based architectures, change is at a domain level

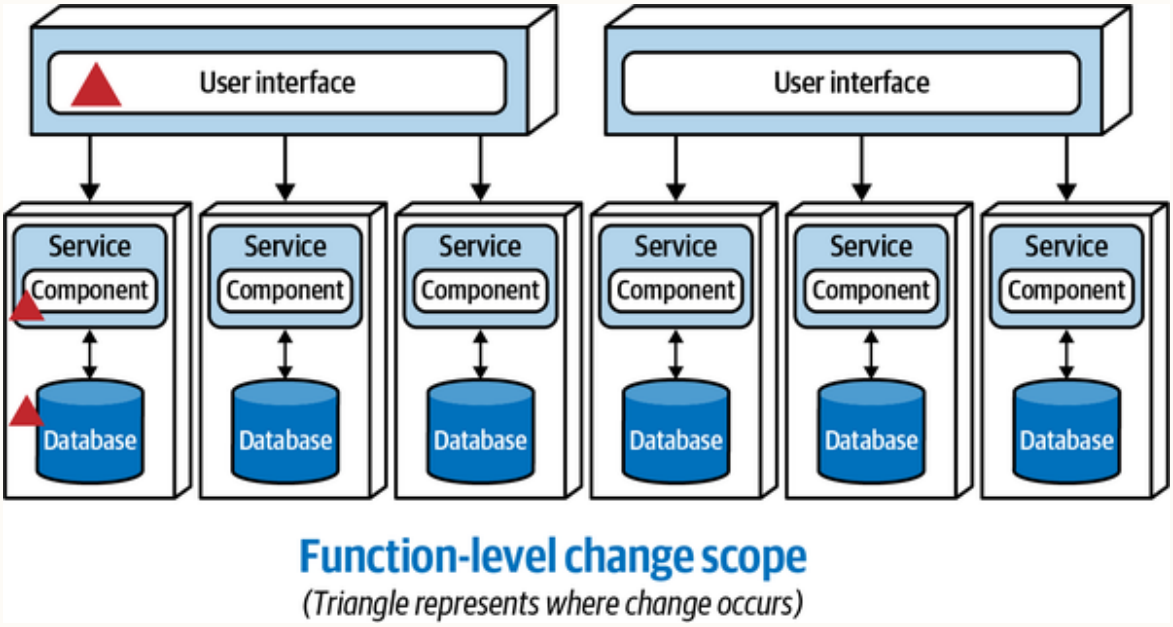


Figure 3-6. With microservices architectures, change is at a function level

These three progressions toward modularity demonstrate that as the level of architectural modularity increases, so does maintainability, making it easier to add, change, or remove functionality.

Testability

Testability is defined as the ease of testing (usually implemented through automated tests) as well as the *completeness* of testing. Testability is an essential ingredient for architectural agility. Large monolithic architecture styles like the layered architecture support relatively low levels of testability (and hence agility) due to the difficulty in achieving full and complete regression testing of all features within the large deployment unit. Even if a monolithic application did have a suite of full regression tests, imagine the frustration of having to execute hundreds or even thousands of unit tests for a simple code change. Not only would it take a long time to execute all of the tests, but the poor developer would be stuck researching why dozens of tests failed when in fact the failed tests have nothing to do with the change.

Architectural modularity—the breaking apart of applications into smaller deployment units—significantly reduces the overall testing scope for changes made to a service, allowing for better completeness of testing as well as ease of testing. Not only does modularity result in smaller, more targeted test suites, but maintaining the unit tests becomes easier as well.

While architectural modularity generally improves testability, it can sometimes lead to the same problems that exist with monolithic, single-deployment applications. For example, consider an application that was broken into three smaller self-contained deployment units (services), as depicted in Figure 3-7.

Making a change to Service A limits the testing scope to only that service, since Service B and Service C are not coupled to Service A. However, as communication increases among these services, as shown at the bottom of Figure 3-7, testability declines rapidly because the testing scope for a change to Service A now includes Service B and Service C, therefore impacting both the ease of testing and the completeness of testing.

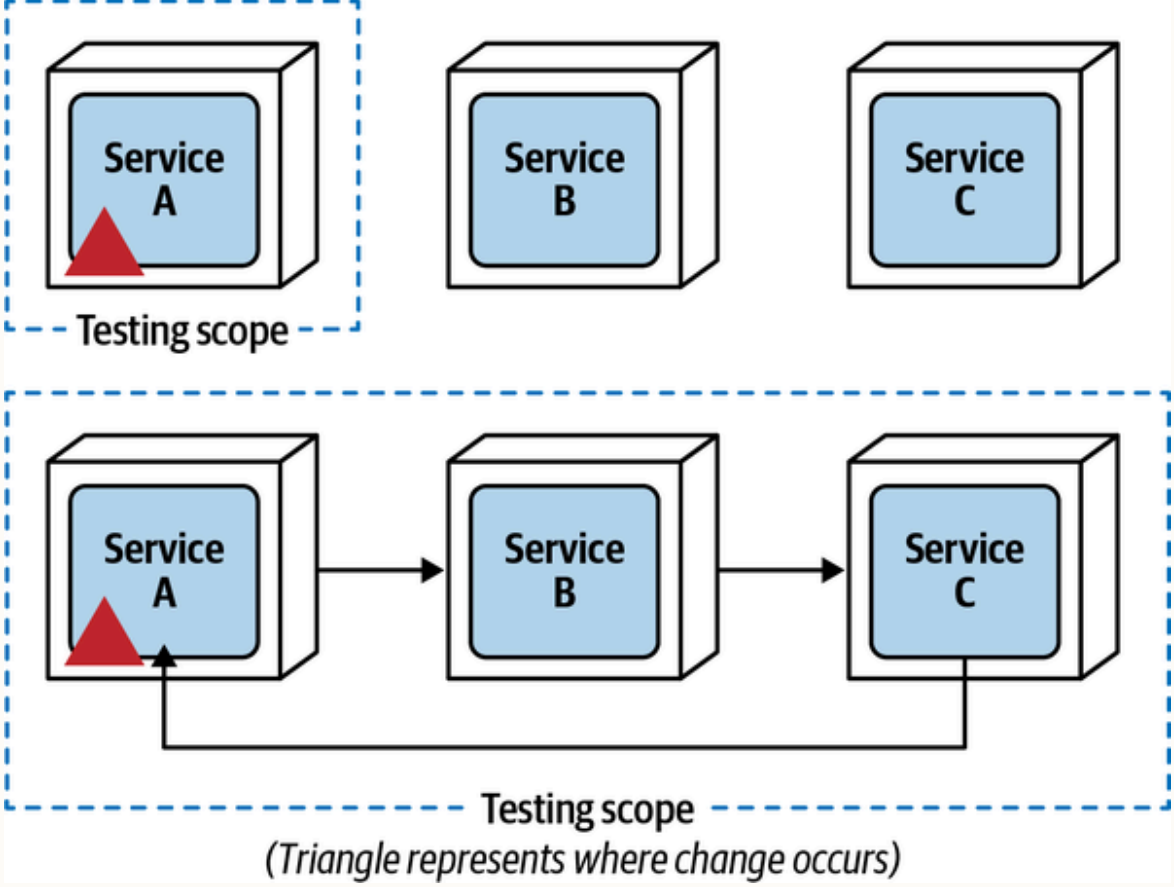


Figure 3-7. Testing scope is increased as services communicate with one another

Deployability

Deployability is not only about the ease of deployment—it is also about the frequency of deployment and the overall risk of deployment. To support agility and respond quickly to change, applications must support all three of these factors. Deploying software every two weeks (or more) not only increases the overall risk of deployment (due to grouping multiple changes together), but in most cases unnecessarily delays new features or bug fixes that are ready to be pushed out to customers. Of course, deployment frequency must be balanced with the customer's (or end user's) ability to be able to absorb changes quickly.

Monolithic architectures generally support low levels of deployability due to the amount of ceremony involved in deploying the application (such as code freezes, mock deployments, and so on), the increased risk that something else might break once new features or bug fixes are deployed, and a long time frame between deployments (weeks to months). Applications having a certain level of architectural modularity in terms of separately deployed units of software have less deployment ceremony, less risk of deployment, and can be deployed more frequently than a large, single monolithic application.

Like testability, deployability is also negatively impacted as services become smaller and communicate more with each other to complete a business transaction. Deployment risk is increased, and it becomes more difficult to deploy a simple change for fear of breaking other services. To quote software architect Matt Stine in his article on orchestrating microservices:

If your microservices must be deployed as a complete set in a specific order, please put them back in a monolith and save yourself some pain.

This scenario leads to what is commonly referred to as the "big ball of distributed mud," where very few (if any) of the benefits of architectural modularity are realized.

Scalability

Scalability is defined as the ability of a system to remain responsive as user load gradually increases over time. Related to scalability is *elasticity*, which is defined as the ability of a system to remain responsive during significantly high instantaneous and erratic spikes in user load. Figure 3-8 illustrates the differences between scalability and elasticity.

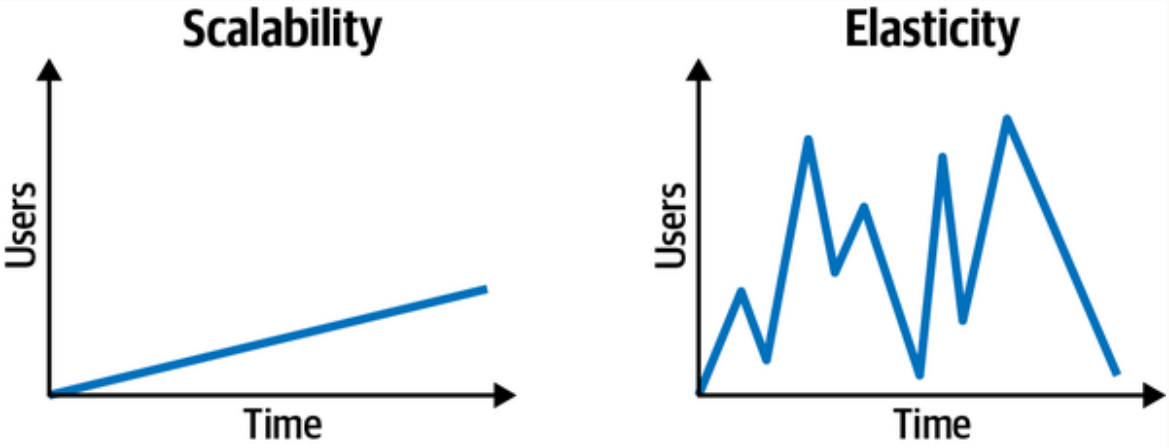


Figure 3-8. Scalability is different from elasticity

While both of these architectural characteristics include responsiveness as a function of the number of concurrent requests (or users in the system), they are handled differently from an architectural and implementation standpoint. Scalability generally occurs over a longer period of time as a function of normal company growth, whereas elasticity is the immediate response to a spike in user load.

A great example to further illustrate the difference is that of a concert-ticketing system. Between major concert events, there is usually a fairly light concurrent user load. However, the minute tickets go on sale for a popular concert, concurrent user load significantly spikes. The system may go from 20 concurrent users to 3,000 concurrent users in a matter of seconds. To maintain responsiveness, the system must have the capacity to handle the high peaks in user load, and also have the ability to instantaneously start up additional services to handle the spike in traffic. Elasticity relies on services having a very small *mean time to startup* (MTTS), which is achieved *architecturally* by having very small, fine-grained services. With an appropriate architectural solution in place, MTTS (and hence elasticity) can then be further managed through design-time techniques such as small lightweight platforms and runtime environments.

Although both scalability and elasticity improve with finer-grained services, elasticity is more a function of granularity (the size of a deployment unit), whereas scalability is more a function of modularity (the breaking apart of applications into separate deployment units). Consider the traditional layered architecture, service-based architecture, and microservices architecture styles and their corresponding star ratings for scalability and elasticity, as illustrated in Figure 3-9 (the details of these architecture styles and their corresponding star ratings can be found in our previous book, *Fundamentals of Software Architecture*. Note that one star means that the capability is not well supported by the architecture style, whereas five stars means that capability is a major feature of the architecture style and is well supported.

Notice that scalability and elasticity rate relatively low with the monolithic layered architecture. Large monolithic layered architectures are both difficult and expensive to scale because all of the application functionality must scale to the same degree (application-level scalability and poor MTTS). This can become particularly costly in cloud-based infrastructures. However, with service-based architecture, notice that scalability improves, but not as much as elasticity. This is because domain services in a service-based architecture are coarse grained and usually contain the entire domain in one deployment unit (such as order processing or warehouse management), and generally have too long of a mean time to startup (MTTS) to respond fast enough to immediate demand for elasticity due to their large size (domain-level scalability and fair MTTS). Notice that with microservices, both scalability and elasticity are maximized because of the small, single-purpose, fine-grained nature of each separately deployed service (function-level scalability and excellent MTTS).

Like testability and deployability, the more services communicate with one other to complete a single business transaction, the greater the negative impact on scalability and elasticity. For this reason, it is important to keep synchronous communication among services to a minimum when requiring high levels of scalability and elasticity.

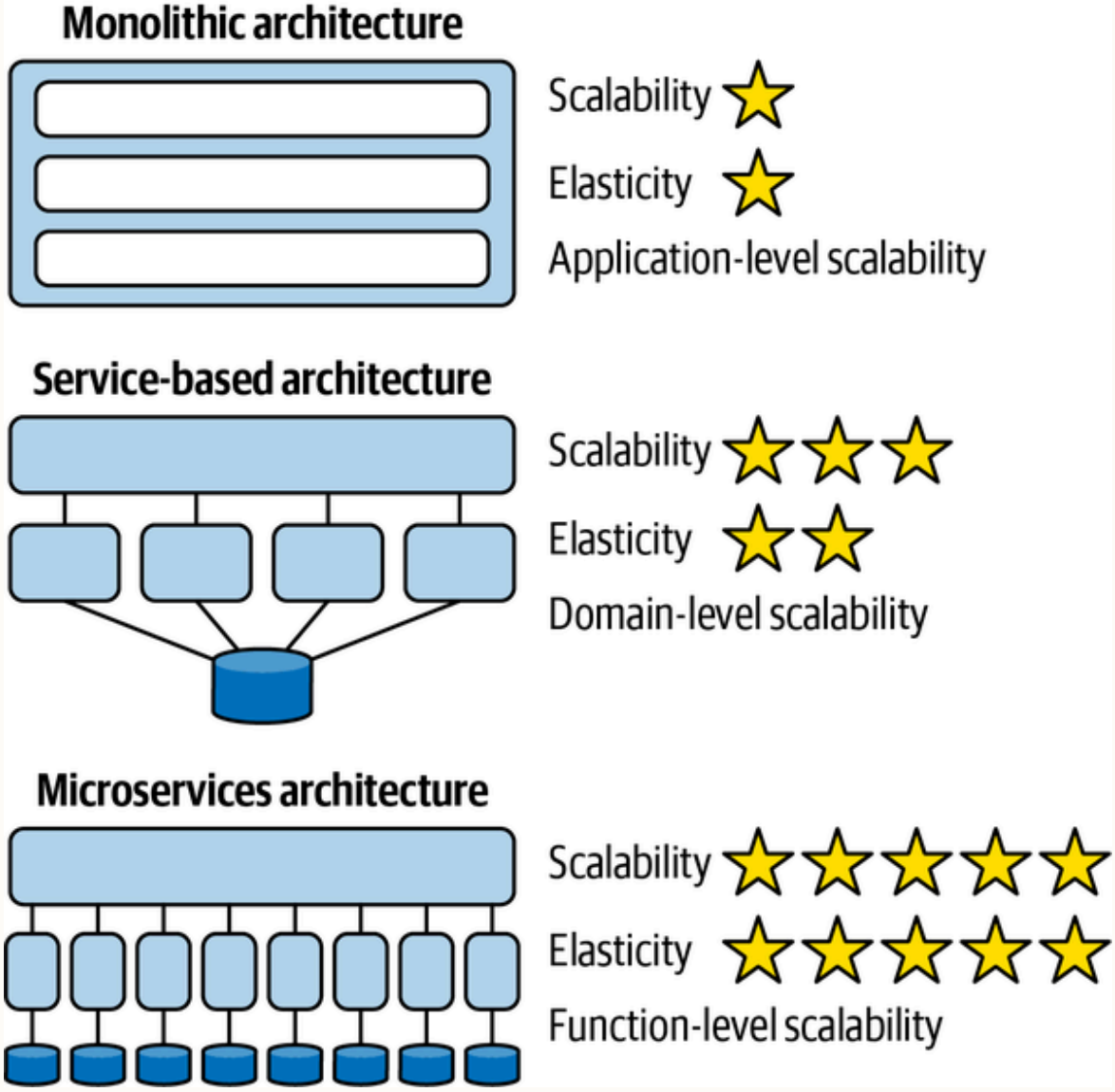


Figure 3-9. Scalability and elasticity improve with modularity

Availability/Fault Tolerance

Like many architecture characteristics, *fault tolerance* has varying definitions. Within the context of architectural modularity, we define fault tolerance as the ability for some parts of the system to remain responsive and available as other parts of the system fail. For example, if a fatal error (such as an out-of-memory condition) in the payment-processing portion of a retail application occurs, the users of the system should still be able to search for items and place orders, even though the payment processing is unavailable.

All monolithic systems suffer from low levels of fault tolerance. While fault tolerance can be somewhat mitigated in a monolithic system by having multiple instances of the entire application load balanced, this technique is both expensive and ineffective. If the fault is due to a programming bug, that bug will exist in both instances, therefore potentially bringing down both instances.

Architectural modularity is essential to achieving domain-level and function-level fault tolerance in a system. By breaking apart the system into multiple deployment units, catastrophic failure is isolated to only that deployment unit, thereby allowing the rest of the

system to function normally. There is a caveat to this, however: if other services are synchronously dependent on a service that is failing, fault tolerance is not achieved. This is one of the reasons asynchronous communication between services is essential for maintaining a good level of fault tolerance in a distributed system.

Sysops Squad Saga: Creating a Business Case

Thursday, September 30, 12:01



Armed with a better understanding of what is meant by *architectural modularity* and the corresponding drivers for breaking apart a system, Addison and Austen met to discuss the Sysops Squad issues and try to match them to modularity drivers in order to build a solid business justification to present to the business sponsors.

"Let's take each of the issues we are facing and see if we can match them to some of the modularity drivers," said Addison. "That way, we can demonstrate to the business that breaking apart the application will in fact address the issues we are facing."

"Good idea," said Austen. "Let's start with the first issue they talked about in the meeting—change. We cannot seem to effectively apply changes to the existing monolithic system without something else breaking. Also, changes take way too long, and testing the changes is a real pain."

"And the developers are constantly complaining that the codebase is too large, and it's difficult to find the right place to apply changes to new features or bug fixes," said Addison.

"OK," said Austen, "so clearly, overall maintainability is a key issue here."

"Right," said Addison. "So, by breaking apart the application, it would not only decouple the code, but it would isolate and partition the functionality into separately deployed services, making it easier for developers to apply changes."

"Testability is another key characteristic related to this problem, but we have that covered already because of all our automated unit tests," said Austen.

"Actually, it's not," replied Addison. "Take a look at this."

Addison showed Austen that over 30% of the test cases are commented out or obsolete, and there are missing test cases for some of the critical workflow parts of the system. Addison also explained that the developers were continually complaining that the entire unit test suite had to be run for any change (big or small), which not only took a long time,

but developers were faced with having to fix issues not related to their change. This was one of the reasons it was taking so long to apply even the simplest of changes.

"Testability is about the ease of testing, but also the completeness of testing," said Addison. "We have neither. By breaking apart the application, we can significantly reduce the scope of testing for changes made to the application, group relevant automated unit tests together, and get better completeness of testing—hence fewer bugs."

"The same is true with deployability," continued Addison. "Because we have a monolithic application, we have to deploy the entire system, even for a small bug fix. Because our deployment risk is so high, Parker insists on doing production releases on a monthly basis. What Parker doesn't understand is that by doing so, we pile multiple changes onto every release, some of which haven't even been tested in conjunction with each other."

"I agree," said Austen, "and besides, the mock deployments and code freezes we do for each release take up valuable time—time we don't have. However, what we're talking about here is not an architecture issue, but purely a deployment pipeline issue."

"I disagree," said Addison. "It's definitely architecture related as well. Think about it for a minute, Austen. If we broke the system into separately deployed services, then a change for any given service would be scoped to that service only. For example, let's say we make yet another change to the ticket assignment process. If that process was a separate service, not only would the testing scope be reduced, but we would significantly reduce the deployment risk. That means we could deploy more frequently with much less ceremony, as well as significantly reduce the number of bugs."

"I see what you mean," said Austen, "and while I agree with you, I still maintain that at some point we will have to modify our current deployment pipeline as well."

Satisfied that breaking apart the Sysops Squad application and moving to a distributed architecture would address the change issues, Addison and Austen moved on to the other business sponsor concerns.

"OK," said Addison, "the other big thing the business sponsors complained about in the meeting was overall customer satisfaction. Sometimes the system isn't available, the system seems to crash at certain times during the day, and we've experienced too many lost tickets and ticket routing issues. It's no wonder customers are starting to cancel their support plans."

"Hold on," said Austen. "I have some latest metrics here that show it's not the core ticketing functionality that keeps bringing the system down, but the customer survey functionality and reporting."

“This is excellent news,” said Addison. “So by breaking apart that functionality of the system into separate services, we can isolate those faults, keeping the core ticketing functionality operational. That’s a good justification in and of itself!”

“Exactly,” said Austen. “So, we are in agreement then that overall availability through fault tolerance will address the application not always being available for the customers since they only interact with the ticketing portion of the system.”

“But what about the system freezing up?” asked Addison. “How do we justify that part with breaking up the application?”

“It just so happens I asked Sydney from the Sysops Squad development team to run some analysis for me regarding exactly that issue,” said Austen. “It turns out that it is a combination of two things. First, whenever we have more than 25 customers creating tickets at the same time, the system freezes. But, check this out—whenever they run the operational reports during the day when customers are entering problem tickets, the system also freezes up.”

“So,” said Addison, “it appears we have both a scalability and a database load issue here.”

“Exactly!” Austen said. “And get this—by breaking up the application *and* the monolithic database, we can segregate reporting into its own system and also provide the added scalability for the customer-facing ticketing functionality.”

Satisfied that they had a good business case to present to the business sponsors and confident that this was the right approach for saving this business line, Addison created an Architecture Decision Record (ADR) for the decision to break apart the system and create a corresponding business case presentation for the business sponsors.

ADR: Migrate Sysops Squad Application to a Distributed Architecture

Context
The Sysops Squad is currently a monolithic problem ticket application that supports many different business functions related to problem tickets, including customer registration, problem ticket entry and processing, operations and analytical reporting, billing and payment processing, and various administrative maintenance functions. The current application has numerous issues involving scalability, availability, and maintainability.

Decision
We will migrate the existing monolithic Sysops Squad application to a distributed architecture. Moving to a distributed architecture will accomplish the following:

- *Make the core ticketing functionality more available for our external customers, therefore providing better fault tolerance*
- *Provide better scalability for customer growth and ticket creation, resolving the frequent application freeze-ups we've been experiencing*
- *Separate the reporting functionality and reporting load on the database, resolving the frequent application freeze-ups we've been experiencing*
- *Allow teams to implement new features and fix bugs much faster than with the current monolithic application, therefore providing for better overall agility*
- *Reduce the amount of bugs introduced into the system when changes occur, therefore providing better testability*
- *Allow us to deploy new features and bug fixes at a much faster rate (weekly or even daily), therefore providing better deployability*

Consequences
The migration effort will cause delays for new features being introduced since most of the developers will be needed for the architecture migration.

The migration effort will incur additional cost (cost estimates to be determined).

Until the existing deployment pipeline is modified, release engineers will have to manage the release and monitoring of multiple deployment units.

The migration effort will require us to break apart the monolithic database.

Addison and Austen met with the business sponsors for the Sysops Squad problem ticketing system and presented their case in a clear and concise manner. The business sponsors were pleased with the presentation and agreed with the approach, informing Addison and Austen to move forward with the migration.

Previous chapter
< 2. Discerning Coupling in Software Architecture

Next chapter
4. Architectural Decomposition >